DATA STRUCTURES ASSIGNMENT

Name : Mahalakshmi S

Register No : 1965260054

Course Name : Data Structures

Course Code : CSA0304

Questions

1. Abstract Data Types (ADTs) in Distributed Systems
2. Recursive Tree Traversal Algorithm
3. Big-Theta (Θ) and Big-Omega (Ω) Notations

Question 1: Abstract Data Types (ADTs) in Distributed Systems

1. Introduction

An Abstract Data Type (ADT) defines a data structure by its behaviour: the set of operations it supports and the rules those operations follow, without revealing how it is implemented. Examples are Stack, Queue, List, Set, Map and Counter. A distributed system is a collection of independent computers (nodes) that communicate over a network and appear to the user as one coherent system. In a distributed system, an ADT must be shared, replicated and kept consistent across many nodes, which raises new design challenges.

2. Why ADTs Matter in Distributed Systems

- **Abstraction:** clients use operations such as put(key, value) or enqueue(x) without knowing which node stores the data or how it is replicated.
- **Encapsulation:** the internal representation (partitioning, replication, storage format) can change without affecting client code.
- **Location transparency:** the same interface works whether the data is local or on a remote node.
- **Modularity and reuse:** the same ADT can be backed by different implementations, for example a Map backed by a hash table on one node or a sharded store across a cluster.

3. Common Distributed ADTs

ADT	Distributed form	Real-world example
Map / Dictionary	Distributed hash table (DHT), key-value store	Cassandra, DynamoDB, Redis Cluster
Queue	Distributed message queue	Apache Kafka, RabbitMQ, Amazon SQS
Set	Replicated set with add/remove merging	Redis sets, CRDT sets
Counter	Replicated or CRDT counter	Like counts, view counters
Lock	Distributed lock / mutex	ZooKeeper, etcd
Log	Replicated append-only log	Raft and Paxos logs, Kafka topics

4. Key Design Issues

(a) Replication

Copies of the data are stored on several nodes to improve availability and read performance. Replicas must be kept in agreement when operations such as add or remove are applied.

(b) Partitioning (Sharding)

Large ADTs such as maps are split across nodes using hashing or ranges. Consistent hashing is widely used so that adding or removing a node moves only a small part of the data.

(c) Consistency

- **Strong consistency (linearizability):** every operation appears to take effect instantly in a single global order. Easier to reason about but slower and less available.
- **Eventual consistency:** replicas may differ temporarily but converge when updates stop. Faster and more available.

(d) Concurrency control

Many clients may call operations at the same time. Locks, optimistic concurrency control, or consensus protocols (Paxos, Raft) ensure operations apply in a safe order.

(e) Fault tolerance

Nodes and network links can fail. The ADT must keep working, using replication, failure detection, leader election and recovery.

(f) Latency and network cost

Each remote operation costs a network round trip, so interface design (for example batching operations) affects performance.

5. The CAP Theorem

A distributed data store can provide at most two of the following three guarantees at the same time during a network partition:

- **Consistency (C):** every read sees the most recent write.
- **Availability (A):** every request receives a response.
- **Partition tolerance (P):** the system continues to work despite lost messages between nodes.

Because partitions cannot be avoided in real networks, designers choose between CP (consistent but may reject requests) and AP (available but may return stale data) behaviour for each ADT.

6. Conflict-Free Replicated Data Types (CRDTs)

CRDTs are ADTs designed so that replicas can be updated independently and concurrently, and always merge to the same state without coordination. They give eventual consistency without locks.

- **G-Counter (grow-only counter):** each node keeps its own count; the value is the sum, and merging takes the maximum per node.
- **PN-Counter:** two G-Counters, one for increments and one for decrements.
- **G-Set and OR-Set:** sets whose add and remove operations commute so that merge order does not matter.

Merge must be commutative, associative and idempotent so that duplicate or reordered messages do not change the result.

7. Example: Distributed Queue

A queue ADT has enqueue(x), dequeue() and isEmpty(). In a distributed message queue:

- Messages are stored in partitions spread over several brokers and replicated for fault tolerance.
- Producers call enqueue; consumers call dequeue and acknowledge messages.
- Ordering is usually guaranteed only within a partition, not across the whole queue, which is a trade-off for scalability.
- Delivery semantics (at-most-once, at-least-once, exactly-once) define how failures are handled.

8. Example: Distributed Key-Value Map

```
interface Map<K, V>:
    put(key, value)
    get(key) -> value
    delete(key)
# Client sees one map. Behind the interface:
# - hash(key) selects the partition (shard)
# - the shard is replicated on N nodes
# - a quorum rule (R + W > N) gives strong reads
```

With N replicas, a write waits for W acknowledgements and a read contacts R replicas. If R + W > N, every read overlaps with the latest write and sees the most recent value.

9. Advantages and Challenges

Advantages	Challenges
Scalability by adding more nodes	Maintaining consistency across replicas
High availability and fault tolerance	Network latency and partitions
Clean interface hides distribution details	Complex failure handling and debugging
Implementations can be swapped without changing clients	Trade-offs between consistency, availability and performance

10. Applications

- Cloud databases and key-value stores (DynamoDB, Cassandra).
- Messaging and event streaming platforms (Kafka, SQS).
- Collaborative editing tools using CRDTs.
- Coordination services providing locks and configuration (ZooKeeper, etcd).
- Distributed caches such as Redis and Memcached.

11. Conclusion

ADTs let distributed systems expose simple, uniform interfaces while hiding replication, partitioning and fault handling. The main design task is to balance consistency, availability and performance, using tools such as quorums, consensus protocols and CRDTs.

Question 2: Recursive Tree Traversal Algorithm

1. Introduction

A tree is a non-linear data structure made of nodes connected by edges. Traversal means visiting every node exactly once in a systematic order. A binary tree is naturally recursive: each node has a left subtree and a right subtree, and each subtree is itself a binary tree. This makes recursion the simplest way to traverse it.

2. Design Approach

- **Base case:** if the current node is null (empty subtree), return without doing anything.
- **Recursive case:** process the current node and recursively traverse the left and right subtrees.
- The order in which the node is processed relative to its subtrees defines the traversal type.

3. Node Structure

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

4. Recursive Algorithms

(a) Preorder: Root, Left, Right

```
Algorithm Preorder(node)
    if node = null then return
    visit(node)
    Preorder(node.left)
    Preorder(node.right)
```

(b) Inorder: Left, Root, Right

```
Algorithm Inorder(node)
    if node = null then return
    Inorder(node.left)
    visit(node)
    Inorder(node.right)
```

(c) Postorder: Left, Right, Root

```
Algorithm Postorder(node)
    if node = null then return
    Postorder(node.left)
    Postorder(node.right)
    visit(node)
```

5. Python Implementation

```
def preorder(n):
    if n is None:
```

```
            return
        print(n.key, end=" ")
        preorder(n.left)
        preorder(n.right)

def inorder(n):
        if n is None:
            return
        inorder(n.left)
        print(n.key, end=" ")
        inorder(n.right)

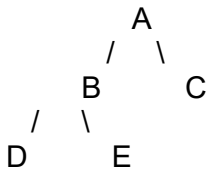
def postorder(n):
        if n is None:
            return
        postorder(n.left)
        postorder(n.right)
        print(n.key, end=" ")

# Build the example tree
root = Node("A")
root.left = Node("B")
root.right = Node("C")
root.left.left = Node("D")
root.left.right = Node("E")

preorder(root)   # A B D E C
inorder(root)    # D B E A C
postorder(root)  # D E B C A
```

6. Worked Example

Consider the tree below:



Traversal	Order	Output
Preorder	Root, Left, Right	A B D E C
Inorder	Left, Root, Right	D B E A C
Postorder	Left, Right, Root	D E B C A

Preorder trace

- Visit A, then go left to B.
- Visit B, then go left to D. Visit D; both children are null, so return.
- Back at B, go right to E. Visit E and return.

- Back at A, go right to C. Visit C and finish.

Output: A B D E C.

7. Correctness

By induction on the number of nodes: an empty tree is handled by the base case. For a tree with n nodes, the left and right subtrees have fewer than n nodes, so by the induction hypothesis they are traversed correctly. The root is visited exactly once, so every node is visited exactly once.

8. Complexity Analysis

Measure	Complexity	Reason
Time	O(n)	Each of the n nodes is visited once with constant work; there are n + 1 calls on null children
Space (balanced tree)	O(log n)	Recursion depth equals the tree height h = log n
Space (skewed tree)	O(n)	Height can be n, so the call stack holds n frames

In general the space complexity is O(h), where h is the height of the tree. The recurrence is $T(n) = T(k) + T(n - k - 1) + O(1)$, which solves to $\Theta(n)$.

9. Applications

- **Preorder:** copying a tree, prefix expressions, serializing a tree.
- **Inorder:** gives keys in sorted order for a binary search tree.
- **Postorder:** deleting a tree, evaluating postfix expressions, computing folder sizes.

10. Recursive vs Iterative Traversal

- Recursive code is shorter and mirrors the structure of the tree.
- Very deep trees can cause stack overflow; an iterative version with an explicit stack avoids this.
- Both have the same O(n) time complexity.

11. Conclusion

Recursive tree traversal needs only a base case for the empty tree and two recursive calls on the subtrees. Choosing where to visit the node gives preorder, inorder or postorder. Each runs in O(n) time and O(h) space.

Question 3: Big-Theta (Θ) and Big-Omega (Ω) Notations

1. Introduction

The efficiency of an algorithm is measured by how its running time or memory use grows as the input size n increases. Exact running times depend on the machine, language and compiler, so we use asymptotic notations, which describe growth for large n while ignoring constant factors and lower-order terms. The three main notations are Big-O (upper bound), Big-Omega (lower bound) and Big-Theta (tight bound).

2. Big-Omega Notation (Ω): Lower Bound

Formal definition

A function f(n) is Ω(g(n)) if there exist positive constants c and n_0 such that:

$$\mathbf{f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0}$$

Explanation

- For large n, f(n) never grows slower than a constant multiple of g(n).
- It is used for best-case analysis and to state the minimum work an algorithm or problem needs.
- Example of a problem bound: any comparison-based sorting algorithm needs Ω(n log n) comparisons in the worst case.

Worked example

Show that $f(n) = 3n^2 + 2n$ is $\Omega(n^2)$.

For all $n \geq 1$, $3n^2 + 2n \geq 3n^2$. Choose $c = 3$ and $n_0 = 1$. The condition holds, so $f(n) = \Omega(n^2)$.

3. Big-Theta Notation (Θ): Tight Bound

Formal definition

A function f(n) is Θ(g(n)) if there exist positive constants c_1, c_2 and n_0 such that:

$$\mathbf{c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0}$$

Explanation

- f(n) is sandwiched between two constant multiples of g(n), so both grow at the same rate.
- f(n) = Θ(g(n)) if and only if f(n) = O(g(n)) and f(n) = Ω(g(n)).
- It is the most precise notation and is used when the best and worst cases have the same growth rate.

Worked example

Show that $f(n) = 3n^2 + 2n$ is $\Theta(n^2)$.

- Lower bound: $3n^2 + 2n \geq 3n^2$ for $n \geq 1$, so $c_1 = 3$.
- Upper bound: $3n^2 + 2n \leq 3n^2 + 2n^2 = 5n^2$ for $n \geq 1$, so $c_2 = 5$.

With $c_1 = 3$, $c_2 = 5$ and $n_0 = 1$, we get $3n^2 \le f(n) \le 5n^2$, hence $f(n) = \Theta(n^2)$.

Second example (disproof)

Is $n^3 = \Theta(n^2)$? No. The upper bound would need $n^3 \le c_2 \cdot n^2$ for all large n, i.e. $n \le c_2$, which fails for $n > c_2$. So $n^3 = \Omega(n^2)$ but $n^3 \neq \Theta(n^2)$.

4. Comparison of Asymptotic Notations

Notation	Bound type	Condition	Typical use
$O(g(n))$	Upper bound	$f(n) \le c \cdot g(n)$	Worst case
$\Omega(g(n))$	Lower bound	$f(n) \ge c \cdot g(n)$	Best case
$\Theta(g(n))$	Tight bound	$c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$	Exact growth rate

5. Important Properties

- **Reflexive:** $f(n) = \Theta(f(n))$.
- **Symmetric:** $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$.
- **Transitive:** if $f = \Theta(g)$ and $g = \Theta(h)$, then $f = \Theta(h)$. The same holds for O and Ω.
- **Transpose symmetry:** $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$.
- **Polynomials:** a polynomial of degree d is $\Theta(n^d)$.

6. Common Growth Rates (slowest to fastest)

Order	Name	Example
$\Theta(1)$	Constant	Array access by index
$\Theta(\log n)$	Logarithmic	Binary search
$\Theta(n)$	Linear	Linear search
$\Theta(n \log n)$	Linearithmic	Merge sort
$\Theta(n^2)$	Quadratic	Bubble sort (worst case)
$\Theta(2^n)$	Exponential	Naive recursive Fibonacci

7. Algorithm Example

Linear search on n elements: in the best case the target is the first element, so the time is $\Omega(1)$. In the worst case all n elements are checked, so the time is $O(n)$. Because the best and worst cases differ, the running time for all inputs cannot be described by one Θ bound. Merge sort, in contrast, always divides and merges the same way, so its time is $\Theta(n \log n)$ in every case.

8. Conclusion

Big-Omega gives a guaranteed minimum growth rate, and Big-Theta pins the growth rate down from both sides. Together with Big-O, they let us compare algorithms independently of hardware and choose the most efficient one for large inputs.